

PaleoRigor: expert-guided workflows for traceable paleomicrobiome data processing

Author names to be completed

Affiliations to be completed

These authors contributed equally, if applicable.

*Correspondence should be addressed to: [corresponding author email]

Keywords: paleomicrobiome; ancient DNA; PaleoRigor; human-in-the-loop analysis; source-data verification; reproducible bioinformatics

Abstract

Background: Ancient microbial evidence informs research on past health, diet, and environments, but its scarcity and susceptibility to contamination demand careful analysis. Paleobiologists need accessible tools to check file selection and processing decisions while retaining expert oversight.

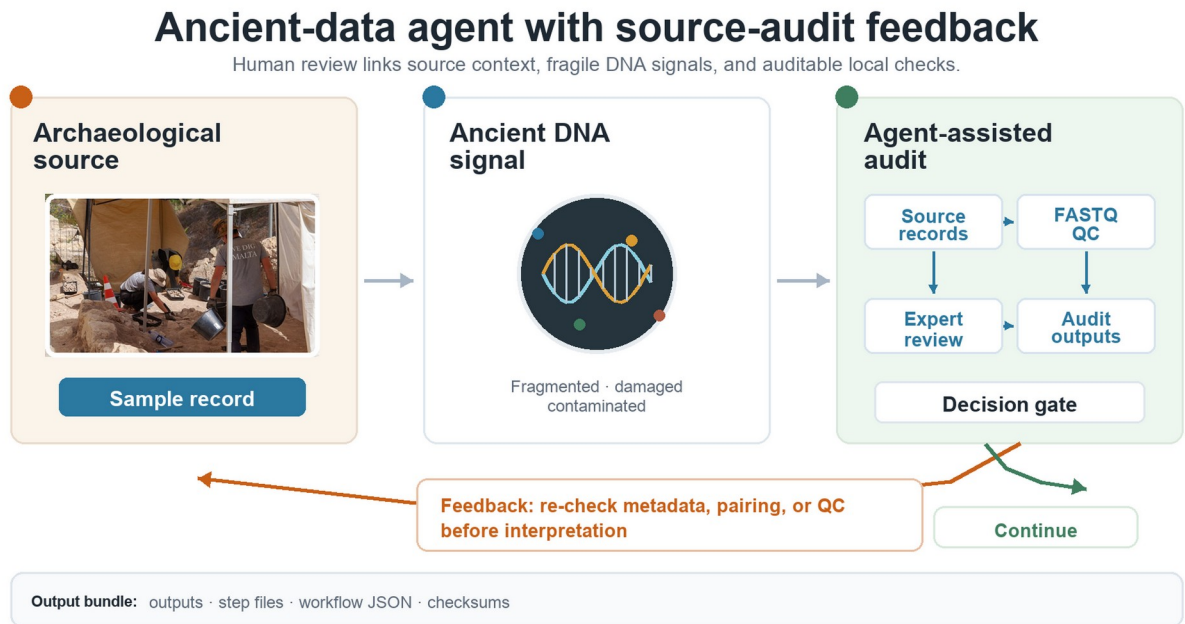
Results: We developed PaleoRigor, a system that translates research requests into workflows built from predefined local analysis modules and reviewed by the user. It checks inputs and proposed operations, requires approval before execution, and retains intermediate files for review. In a frozen held-out evaluation, PaleoRigor passed 23 of 24 runs (95.8%; Wilson 95% confidence interval, 79.8–99.3%). An ablation that removed the PaleoRigor planning contract from the same model passed 19 of 24 (79.2%); PaleoRigor handled all 12 prespecified boundary decisions correctly. Repeating the frozen design with a second model configuration yielded 24 of 24 successes for PaleoRigor and 19 of 24 for the ablated arm. Across both configurations, PaleoRigor passed 47 of 48 runs (97.9%), compared with 38 of 48 (79.2%) for the ablated arms. Dataset-level tests matched read counts and compressed-file sizes for six public sequencing records and documented the removal of 114 duplicate table rows. An audit of a retraction-associated record also illustrated how expert review limited conclusions to the file-level evidence.

Conclusions: PaleoRigor supports workflow review and traceable data handling before specialist interpretation, with an Apple Silicon macOS application providing the tools for local use. It helps paleobiologists inspect computational decisions while leaving ancient-DNA authentication and biological interpretation to specialist assessment.

Key points

- PaleoRigor addresses four risks that precede paleobiological interpretation: ambiguous requests, incorrect source files, undocumented data changes, and conclusions unsupported by the available evidence.
- It makes these risks reviewable by linking each request to its inputs, approved operations, intermediate files, and expert decisions.
- In the final held-out evaluation, PaleoRigor passed 23 of 24 runs (95.8%), compared with 19 of 24 (79.2%) for the ablated arm; all 12 boundary decisions were handled correctly.

Graphical abstract



Graphical abstract. PaleoRigor connects archaeological source material, fragile ancient biomolecular evidence, and reviewable local analysis. The term agent-assisted audit in the figure refers to PaleoRigor; it does not imply autonomous biological interpretation. The archaeological excavation photograph was adapted from Godwin Borg, Wikimedia Commons, CC BY-SA 4.0; all other schematic elements were created by the authors. The graphical abstract shows the paleobiological use case, while Figure 1 shows the expert review and local execution steps.

Background

Paleomicrobiome and ancient-human microbiome studies examine archaeological material to investigate past health, diet, and microbial exposure [1–5]. Sequencing data are often deposited in the Sequence Read Archive (SRA) and European Nucleotide Archive (ENA) [6, 7]. Access to these data is only the starting point for analysis. Researchers must select the intended files, check their local copies, choose appropriate operations, and document subsequent changes. Errors at this stage can persist through an otherwise successful analysis. A traceable connection between the research question, repository record, and local workflow is therefore essential (Figure 1; Table 4).

These requirements are particularly consequential in ancient biomolecular research, where DNA is often scarce, fragmented, damaged, and contaminated. Authenticity depends on several sources of evidence, including sample history, molecular damage, contamination controls, and archaeological context [3, 8–12]. Mapping, adapter removal, and damage analysis assess the files supplied to them [13–15]; they cannot compensate for an incorrect starting file. Source verification therefore supports, but does not replace, biological authentication. Figure 4 evaluates agreement with public records, and Figure 5 illustrates the limits of conclusions drawn from that agreement.

Standardized workflows can reduce manual handling and make complex analyses easier to review. Workflow engines support reproducibility once users have specified inputs and operations [16–18].

Determining whether those choices suit the research question still requires expertise. Natural-language agents offer a way to connect requests with analysis tools [19], but their plans can contain nonexistent files, incompatible operations, or unsupported interpretations. Researchers therefore need a way to inspect and constrain a proposed analysis before it runs, while retaining responsibility for the biological conclusions.

We developed PaleoRigor to support this review process. Its planner converts a research request into a proposed workflow, and its validator checks file references, formats, and compatibility between steps. Each skill is a predefined local analysis module that the user can inspect. After approval, the executor runs the selected skills and saves their intermediate and final outputs. A report links those outputs to the workflow and its execution records. This division allows paleobiologists to review the proposed analysis and assess its results without delegating scientific judgment to the planning model (Figure 1; Tables 1 and 2).

We evaluated workflow execution, data traceability, and the limits of interpretation. Repeated benchmark runs tested whether supported requests produced the required outputs and whether unsuitable requests were blocked for the correct reason. Dataset-level cases then examined a recorded table transformation, agreement between sequencing files and public records, and an expert-defined audit of a retraction-associated record. Together, these tests assess preparation for specialist analysis, rather than ancient-DNA authentication or biological reconstruction.

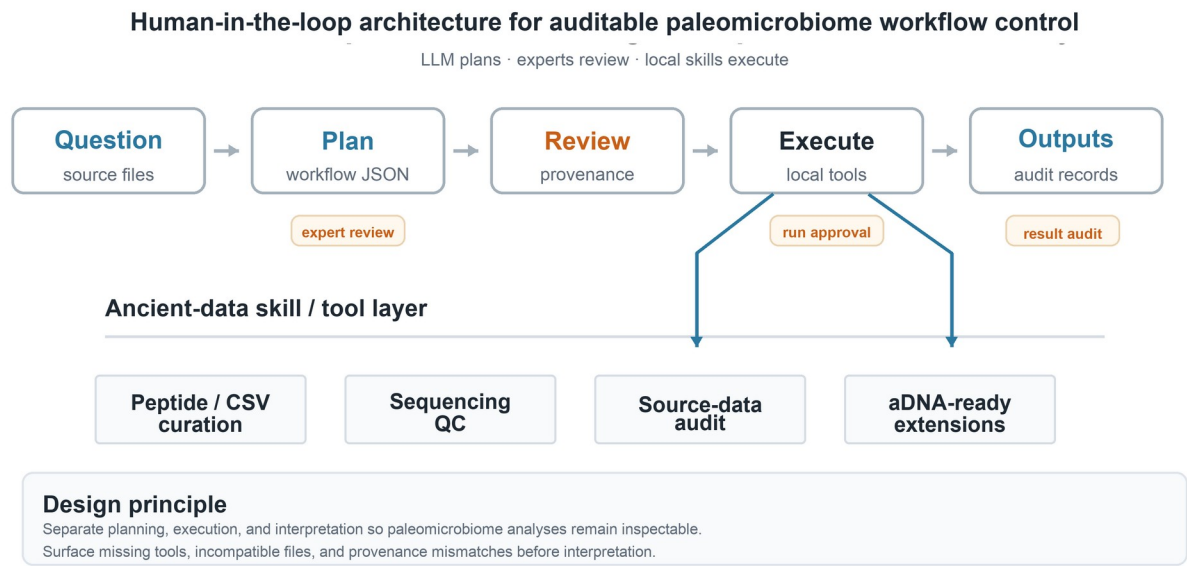


Figure 1. Human-in-the-loop design of PaleoRigor. A paleobiologist starts with a research question and available source material, such as metadata tables, sequencing files, or repository records. PaleoRigor drafts a structured workflow. Experts then review the source, file organization, task fit, and parameters before local execution. Approved skills perform limited tasks such as table cleaning, sequencing quality control, source-record comparison, and evidence packaging. Final outputs are stored separately from step records so that readers can inspect both the result and how it was produced. In the figure, LLM means large language model, and workflow JSON means a machine-readable text record of the proposed plan.

Figure 1 identifies where paleobiologists remain responsible for decisions: defining the question, approving the proposed workflow, and interpreting the retained evidence. Supplementary Table S4 links these checkpoints to the errors they are intended to prevent.

Results

Development failures guided four workflow safeguards

Development revealed four recurring problems: unsuitable requests, incompatible files or references, undocumented transformations, and conclusions unsupported by the evidence. PaleoRigor passed 18 of 24 runs in v3 and 18 of 24 in v4 (75.0% in each round), with 12 failures overall. Specific failures included mismatched output names, unrecognized chart aliases, and unnecessary sample-sheet planning when the referenced FASTQ files were unavailable.

We addressed these problems through expert review, file and workflow compatibility checks, retained intermediate outputs, and explicit limits on interpretation (Figure 1; Table 1). After fixing the identified defects and freezing the revised version, v5 passed 23 of 24 runs and handled all 12 boundary decisions correctly (95.8% overall). The remaining failure involved an uploaded-file reference without its .tsv suffix. The validator detected the unresolved reference and stopped the workflow before execution.

Success increased from 18 of 24 runs in v4 to 23 of 24 in v5, a difference of 20.8 percentage points. Because the earlier rounds informed development, this comparison describes improvement during development rather than an independent estimate of effectiveness. The frozen v5 comparison below evaluates the revised system against an ablation of its own planning contract.

Table 1. PaleoRigor control layers and expert checkpoints. The table shows how planning, validation, execution, output storage, and interpretation are kept separate. This design supports auditable preparation for paleomicrobiome analysis without replacing expert judgment.

Layer	Risk addressed	Control used	Expert intervention
Planner	Ambiguous user intent	Structured workflow JSON	Review proposed steps before running
Validator	Missing outputs or wrong file type	Reference and compatibility checks	Reject or revise invalid workflow
Executor	Uncontrolled code generation	Predefined skills and local tools	Approve local execution
Reporter	Opaque outputs	Final files, step files, records and checksums	Interpret results and boundaries

Prespecified criteria connected workflow checks to retained evidence

Table 2 defines six criteria covering workflow validity, file compatibility, source agreement, traceability, software dependencies, and limits on permitted analyses. A supported v5 run passed only if it met every

applicable planning, validation, and execution requirement. A boundary run passed only if it stopped with the prespecified reason. PaleoRigor passed 23 of 24 runs: 11 of 12 supported workflows and 12 of 12 boundary decisions. We recomputed all 48 outcomes from retained records across both arms; all 48 matched the stored labels.

The dataset-level evaluations assessed source agreement, data changes, and traceability. All six public sequencing records matched repository read counts exactly ($\Delta\text{reads} = 0$), with compressed-file-size ratios of $\text{Rbytes} = 1.000$. Table curation reduced 5,810 rows to 5,696 unique records and documented all 114 removed duplicates (1.96% of input rows). Each case retained final outputs, intermediate files, and run records. These checks established how the inputs were processed and compared with their source records; they did not establish ancient-DNA authenticity.

Table 2. Engineering tests used to assess PaleoRigor. Each test represents an error seen during development and states how the system should prevent or expose it before biological interpretation. Δreads is the local read count minus the public read count. Rbytes is the local compressed FASTQ size divided by the public `fastq_bytes` value.

Evaluation target	Metric or rule	Interpretation
Workflow validity	All step inputs must reference uploaded files or earlier outputs	Prevents hallucinated pipeline references
File compatibility	Input format must match skill requirements	Prevents applying FASTQ operations to CSV files
Source agreement	$\Delta\text{reads} = 0$ and $\text{Rbytes} = 1$ when public metadata are available	Confirms local files match repository records
Traceability	Final outputs plus step outputs and run records	Allows later inspection by users or supervisors
Dependency behavior	Missing tools are reported instead of auto-installed	Keeps execution transparent on user machines
Repeated-run workflow success and boundary control	Supported tasks must pass all tests. Incompatible or overreaching requests must stop before unsafe execution.	Separates repeatable workflows from safe scientific boundaries.

A control-layer ablation isolated the contribution of the planning contract

This comparison was an ablation of the PaleoRigor control layer, not a trial against an external tool. Both arms used the same model, task files, workflow schema, blocked-decision schema, uploaded-file summaries, skill catalogue, call-order protocol, execution path and scoring rules; that shared context was byte-identical at 35,682 characters. The arms differed only in the planning instructions preceding it. The ablated arm received a four-sentence planner instruction of 247 characters. The PaleoRigor arm received the full planning contract of 40,347 characters, comprising thirty staged-workflow and format-compatibility rules, one worked example workflow, and six control-layer rules that name the four

boundary reason codes the system is designed to emit. The boundary comparison therefore measures whether an explicitly encoded rule is applied consistently; it does not measure whether a planner identifies a boundary it was not told about. Both system prompts are reproduced verbatim in Supplementary File 4. With V4-Flash, PaleoRigor passed 23 of 24 runs (95.8%), compared with 19 of 24 (79.2%) for the ablated arm, a difference of 16.7 percentage points. Five paired outcomes favored PaleoRigor and one favored the ablated arm (exact McNemar $p = 0.219$). With V4-Pro, the corresponding counts were 24 of 24 and 19 of 24, a difference of 20.8 percentage points. All five discordant pairs favored PaleoRigor ($p = 0.0625$). Across both configurations, the descriptive totals were 47 of 48 successes (97.9%) for PaleoRigor and 38 of 48 (79.2%) for the ablated arms.

Table 3. Operational comparison of PaleoRigor with existing workflow practices. This is not a head-to-head speed or biological-accuracy benchmark. The measured-outcome column reports results obtained for PaleoRigor in this study. Figure 2 provides a control-layer ablation using the same model with the PaleoRigor planning contract removed; the remaining comparison statements describe common differences in workflow control and were not measured as external software trials.

Workflow-control property	Measured PaleoRigor outcome	Comparison interpretation
Planning validity	Workflows were represented as explicit ordered steps with declared inputs, outputs, parameters, and validation rules before local execution.	Unlike manual command lines or unconstrained large-language-model code, the plan can be inspected before anything runs.
Tabular biological data curation	The peptide validation table was curated from 5,810 rows to 5,696 non-redundant records; 114 duplicate sequence entries were removed.	The result is not only a cleaned table, but a recorded transformation with reusable outputs.
Public FASTQ source-metadata correspondence	Across six public FASTQ records, PaleoRigor-derived read counts matched ENA/SRA metadata with $\Delta\text{reads} = 0$, and compressed-file ratios were $R_{\text{bytes}} = 1.000$.	The system quantifies correspondence with intended public records at the tested metadata level before downstream interpretation.
Retraction-associated source audit	For SRR29088443, PaleoRigor generated an auditable source-level report while limiting interpretation to file-level reproducibility.	The audit separates source-data reproducibility from disputed or retracted biological claims.
Output traceability	Each local run exports final_outputs, step_outputs, and ResearchAgent Records containing workflow JSON, manifests, checksums, timestamps, and run reports.	The evidence chain can be reviewed later by users, supervisors, or domain experts.

Workflow-control property	Measured PaleoRigor outcome	Comparison interpretation
Repeated-run engineering success	Final v5: PaleoRigor 23/24 strict successes (95.8%) versus 19/24 (79.2%) for the ablated control arm; all 12 PaleoRigor boundary decisions passed.	Shows release-level workflow success and boundary control for the fixed task set, while the nonsignificant paired comparison and wide intervals limit generalization.

This ablation assessed the planning contract under otherwise identical execution conditions. It did not compare speed or biological accuracy against an external bioinformatics pipeline, a manual command-line protocol, or an established workflow engine (Table 3). Figure 2 details benchmark performance; Figure 3 traces a table transformation; Figures 4 and 5 examine source agreement and the limits of file-level evidence.

Final held-out evaluation met the prespecified workflow-success threshold

Figure 2A shows the development sequence leading to the final evaluation. Each earlier qualification round comprised four supported workflows and four boundary requests, repeated three times in both study arms. PaleoRigor passed 18 of 24 runs in v3 and 18 of 24 in v4 (75.0% in each round). Failures arose when declared output names did not match skill outputs, chart-name variants were rejected, or sample-sheet validation required unavailable FASTQ files. These rounds informed corrections to output declarations and planning rules before v5 was frozen; their failures remain in the reported record.

V5 used new files, identifiers, values, and request wording, with four supported and four boundary scenarios repeated three times per arm. This produced 24 runs for PaleoRigor and 24 for the ablated control arm. Supported runs had to satisfy every planning, validation, and execution criterion; boundary runs had to return the prespecified reason for blocking execution. PaleoRigor passed 23 of 24 runs (95.8%; Wilson 95% confidence interval, 79.8–99.3%), exceeding the threshold of at least 22 of 24. The ablated arm passed 19 of 24 (79.2%; 59.5–90.8%; Figure 2B).

The paired difference was 16.7 percentage points. Five runs passed only with PaleoRigor, whereas one passed only with the control (exact two-sided McNemar $p = 0.219$). The small comparison did not establish statistical superiority. By task class, PaleoRigor passed 11 of 12 supported workflows (91.7%) and all 12 boundary decisions (100%). The control passed 9 of 12 supported workflows (75.0%) and 10 of 12 boundary decisions (83.3%).

Scenario-level results locate the remaining errors (Figure 2C). PaleoRigor passed 3 of 3 runs for FASTA curation, paired FASTQ quality control, peptide-table processing, and each of the four boundary scenarios. It passed 2 of 3 runs for metadata-to-sample-sheet preparation. In the failed run, the planner omitted the .tsv suffix from an uploaded-file reference, which the validator rejected before execution. The control passed 1 of 3 runs for both FASTA curation and file-format mismatch decisions, and 2 of 3 peptide-table runs.

The frozen v5 release met its prespecified success criterion, with one reference-resolution failure still evident. The earlier 75.0% qualification results remain development evidence rather than independent validation of v5. The benchmark therefore supports the evaluated release and task set; broader reliability requires testing across additional data types, requests, and settings.

We repeated the frozen design with DeepSeek-V4-Pro to assess performance with a second planner. Tasks, files, prompts, validation rules, scoring, tools, call order, and the 24-run sample per arm were unchanged. PaleoRigor passed 24 of 24 Pro runs (100%; Wilson 95% confidence interval, 86.2–100%), compared with 19 of 24 for the ablated arm (79.2%; 59.5–90.8%). The paired difference was 20.8 percentage points; all five discordant pairs favored PaleoRigor (exact two-sided McNemar $p = 0.0625$). This met the preregistered criterion of at least 22 PaleoRigor successes and more successes than the ablated arm. Both configurations therefore met their engineering criteria, although the comparison neither ranked models nor tested generalization across providers.

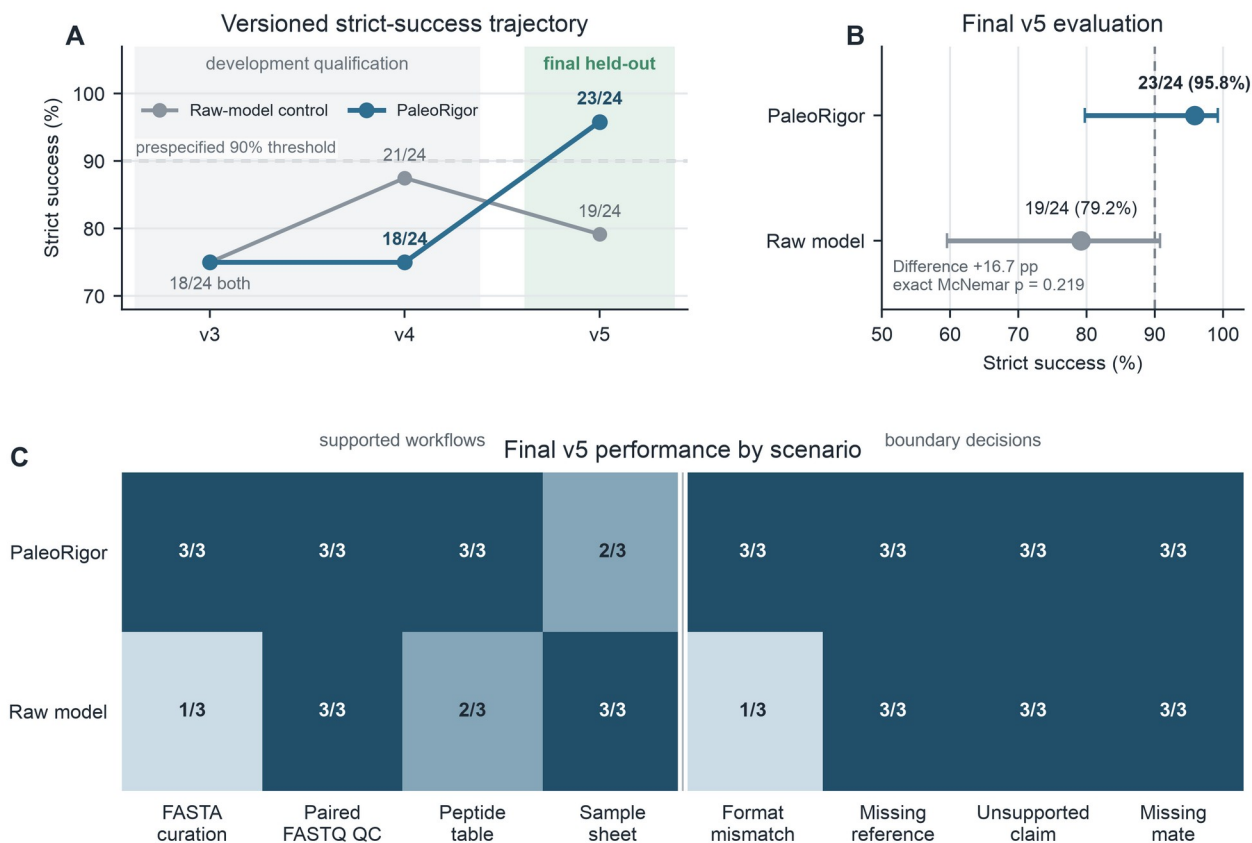


Figure 2. Versioned engineering benchmark and final held-out evaluation. A, Strict run-level success across v3, v4, and v5. V3 and v4 were retained as development-stage qualification rounds; each contained 24 PaleoRigor and 24 ablated-arm runs. The frozen v5 release used new files, values, identifiers, and wording. The dashed line marks the prespecified v5 threshold of more than 90%, operationalized as at least 22 of 24 successful PaleoRigor runs. B, Final v5 strict success for PaleoRigor and for the ablated arm, in which the PaleoRigor planning contract was removed from the same model. Points show observed proportions and horizontal bars show two-sided Wilson 95% confidence intervals. The paired absolute difference was 16.7 percentage points; the exact two-sided McNemar p value was 0.219. C, V5 successes by scenario. Supported workflows comprised FASTA curation, paired FASTQ quality control, peptide-table processing, and sample-sheet preparation. Boundary decisions tested a file-format mismatch, a missing reference, an unsupported authenticity or contamination claim, and a missing paired-end mate. Each cell reports successful runs out of three.

Packaged applications bundled the tools required for local use

We packaged the system as a self-contained Apple Silicon application for macOS 13 or later, including seven analysis tools and their runtimes. In the release gate recorded on 2026-08-23, all 278 project tests in the test suite passed, and all seven bundled commands returned the expected tool identity or version. The application passed strict ad-hoc signature verification and ran from a mounted, read-only disk image. It also removed its temporary launch-token file after use and stopped the local backend when closed. These results verified packaging on the build and test machine; independent usability testing was still required.

A Windows 10/11 x64 installer is produced by a separate native build workflow from the same backend source. It bundles the same seven analysis tools with an Eclipse Temurin 21 runtime and stores model

credentials in Windows Credential Manager; its Samtools build is version 1.24 rather than the 1.23.1 used on macOS. Source URLs and archive checksums for every bundled component are pinned in the repository packaging manifests for both platforms. The Windows build was verified by its automated build and tool-identity checks only; every result reported in this manuscript was produced on macOS.

Case study 1: a domain-neutral test of traceable table curation

The first case tested whether saved records could explain a change to a data table. We used a peptide table because duplicate removal provides a small, exactly checkable transformation. PaleoRigor standardized columns, checked sequence syntax, removed 114 duplicates from 5,810 input rows, and exported 5,696 unique records (Figure 3A,B). The case assessed general data handling rather than biological interpretation.

The run record attributed the removal of 114 rows to the deduplication step. Class counts and peptide-length distributions provided additional checks for unintended changes (Figure 3C,D). This documented transformation satisfied the traceability criterion in Table 2, without testing peptide biology or ancient-microbiome interpretation.

Figure 3 links the processing steps in panel A to the row-count change in panel B. Panels C and D describe the retained label counts and sequence lengths, allowing the exported table to be reviewed alongside its transformation history.

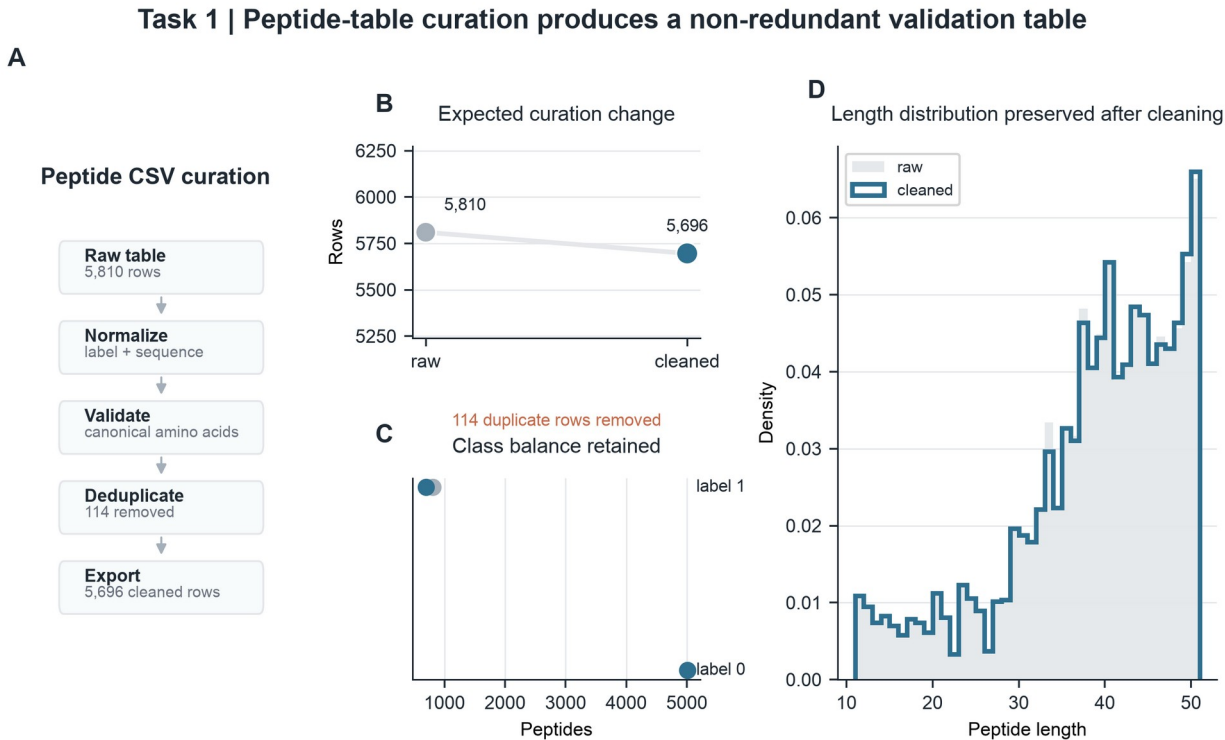


Figure 3. Peptide CSV curation. A, Steps used to process the peptide table. B, Row count before and after duplicate removal, from 5,810 input rows to 5,696 unique records. C, Counts for the two peptide labels before and after cleaning. D, Peptide-length distribution after cleaning, measured as amino-acid residues per sequence.

Case study 2: source-record agreement before ancient-DNA interpretation

The second case tested whether local sequencing files agreed with the intended repository records before ancient-DNA interpretation. Six public records related to ancient or paleomicrobiome research were examined. PaleoRigor checked file structure, ran FastQC, summarized reports with MultiQC, calculated SeqKit statistics, and compared local files with ENA/SRA metadata [6, 7, 20–22]. Records retained the accession, sequencing layout, and mate organization; R1 and R2 identify the two paired-end files.

All six records matched public read counts exactly ($\Delta\text{reads} = 0$), and compressed-file sizes matched repository values ($R\text{bytes} = 1$; Figure 4A,B). These checks support agreement with the intended records at the metadata level. Read counts and sizes cannot establish byte-for-byte identity or ancient-DNA authenticity. They provide an initial check of source selection before downstream processing [13, 15].

Figure 4 distinguishes source comparison from descriptive quality control. Panels A and B compare read counts and file sizes; panel C reports GC content; panel D summarizes record agreement. These measurements describe the files and their relationship to repository metadata, leaving authenticity and contamination to separate analyses.

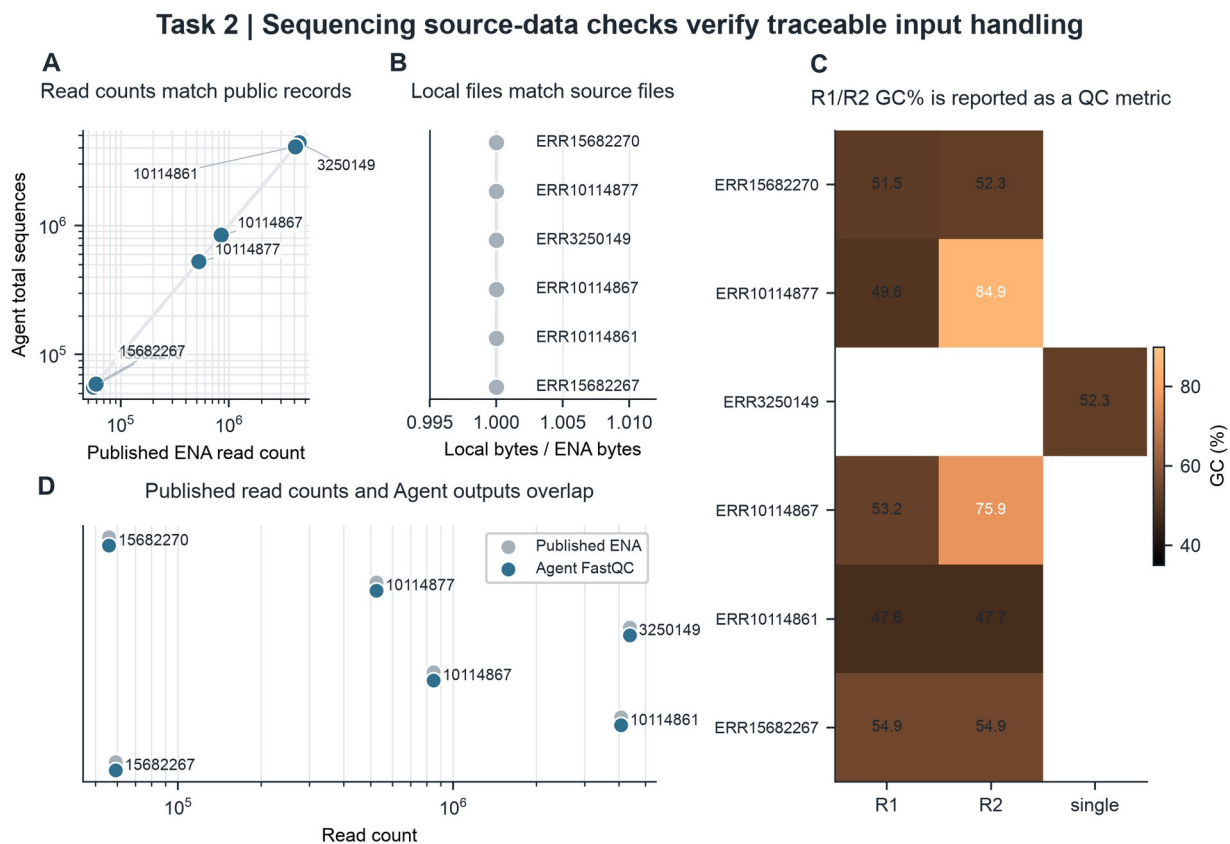


Figure 4. Agreement between local sequencing files and public records. A, Local read counts compared with ENA/SRA read counts. B, Local compressed-file sizes compared with public fastq_bytes values. Δreads is local reads minus public reads. $R\text{bytes}$ is local compressed bytes divided by public fastq_bytes. C, GC content for R1, R2, or single-end files. D, Summary of record agreement across the tested files. Agreement supports a match with the intended public record, but it does not prove byte-level identity, ancient-DNA authenticity, or biological validity.

Case study 3: a negative-control test using a retraction-associated record

The third case examined how expert review could limit conclusions when technically reproducible files were associated with a retracted publication. We selected SRR29088443 from SRP508771 because the SRA study title and submitting center matched the publication record [23, 24]. The accession itself was not marked as retracted. These data originated from a modern clinical microbiome study, so the case tested limits on interpretation rather than ancient-microbiome analysis. The expert restricted the workflow to file-level checks and excluded reassessment of the retracted biological claim (Figure 5A).

Within the approved scope, PaleoRigor reproduced the repository values. Each FASTQ mate contained 53,571 reads, with 26,517,645 bases across both files, and both compressed-file sizes matched the public record. Quality-control summaries reported mean read lengths of 244 and 251 base pairs (bp), with GC contents of 54.0% and 53.6%, respectively (Figure 5B,C). These measurements characterized the available files without testing the biological interpretation of the retracted study.

Figure 5D makes the division of responsibility explicit. The expert supplied the retraction context and defined the permitted analysis; PaleoRigor performed the file checks and retained their outputs. The software did not independently detect or adjudicate the retraction. The case illustrates how expert-defined scope can prevent agreement with repository metadata from being presented as biological validation. The separate boundary tests in Figure 2 assess how reliably the planner followed predefined restrictions.

Task 3 | Boundary-aware audit of a retraction-associated FASTQ record



Figure 5. File-level audit of a retraction-associated FASTQ record. Retraction-associated refers to the linked publication context, not to an autonomous retraction decision by PaleoRigor. The modern clinical microbiome record was used as a negative-control test of interpretation boundaries, not as an ancient-microbiome dataset. A, The expert limited the question to a source audit. B, Public metadata and PaleoRigor outputs matched for R1 reads, R2 reads, total bases, and compressed-file sizes. R1 and R2 are paired-end mates. C, Descriptive quality-control results from the same files, including GC content, mean read length, malformed-record checks, and N-fraction. N-fraction is the proportion of bases recorded as N. D, The allowed conclusion was limited to file-level reproducibility and did not validate the retracted biological interpretation.

Discussion

PaleoRigor addresses a practical problem that arises before specialist interpretation: researchers can lose track of which files were analysed and how those files changed. Incorrect inputs, incompatible operations, and undocumented transformations can all produce plausible-looking outputs. By exposing the proposed workflow and retaining the resulting records, PaleoRigor helps paleobiologists review these decisions before using the outputs to support biological claims.

The benchmark assessed whether this approach supported successful workflows and appropriate blocking decisions. After defects identified in two development rounds were corrected, the frozen v5 release passed 23 of 24 runs (Figure 2A,B). Its remaining error was an unresolved file reference, which the validator blocked before execution. With V4-Pro, PaleoRigor passed 24 of 24 runs, compared with 19

of 24 for the ablated arm. Meeting the criterion with both models supports further evaluation of the approach, but does not establish performance across providers or unfamiliar tasks.

The dataset cases show what traceability contributes beyond a successful run. Across six sequencing records, read counts and compressed-file sizes agreed exactly with repository metadata (Figure 4). In the table case, the retained records explained the removal of 114 rows through a specific deduplication step (Figure 3). These checks help researchers review source selection and data handling before asking whether the material supports an ancient biological signal.

The boundary tests examined when the system should decline a requested analysis. PaleoRigor returned the required blocked decision in all 12 v5 boundary runs, covering incompatible formats, missing references, unsupported claims, and missing mates (Figure 2C). Figure 5 illustrates the complementary role of expert review: an expert supplied publication context and restricted the conclusions allowed from a file audit. Together, these mechanisms address defined computational and interpretive risks, while leaving archaeological meaning and molecular authenticity to specialist assessment.

For paleobiologists, the practical value is a starting point that can be checked before more demanding analyses proceed. Source and workflow review can precede adapter removal, damage analysis, host removal, taxonomic profiling, contamination assessment, and statistical interpretation [13, 15]. Retaining intermediate evidence allows experts to revisit a choice, identify an error, or reject an analysis while preserving its connection to the original question.

Further evaluation should reflect the uncertainty of archaeological samples more closely. Controlled mixtures could include known amounts of endogenous, environmental, handling, and laboratory material. Such tests could assess whether PaleoRigor selects appropriate controls and declines unsupported authenticity claims across contamination levels [25–28]. This would extend evaluation from executing defined tasks to handling uncertainty in ancient biomolecular research.

Additional negative controls would help test the limits of this approach. Retracted or disputed studies and low-biomass datasets affected by reagent contamination offer relevant examples [26, 28]. The present case demonstrates an expert-defined restriction, while the benchmark evaluates responses to predefined boundary categories. Neither establishes how the system would respond to unfamiliar claims, mixed contamination sources, or incomplete archaeological metadata.

Practical adoption also requires accessible software and inspectable records. The repository retains requests, workflows, intermediate files, checksums, reports, figure sources, and environment details (Supplementary Tables S1–S6). The public website [29] explains the method and its safeguards. The Apple Silicon application opens a local browser interface and includes seven analysis tools with their required runtimes. Users can review and run workflows without separately installing Python, Conda, Java, or the bundled tools. Whether this packaging improves usability for paleobiologists remains a question for independent user testing.

Limitations

The present evidence concerns workflow execution, traceable transformations, and agreement with repository metadata. The frozen v5 task set achieved a 95.8% strict-success rate, but this does not establish ancient-DNA authentication, contamination-source identification, or microbial ecological reconstruction. Table 3 compares operational features; the ablated arm evaluates the planning contract rather than biological accuracy against a complete pipeline. The peptide case tests general table handling, and the retraction-associated case uses modern clinical data. The reported evaluation used the Apple Silicon macOS application for macOS 13 or later; a Windows 10/11 x64 build exists but produced no reported result, so cross-platform equivalence and broader usability testing remained outside this evaluation. Larger ancient-data panels, controlled contamination mixtures, and comparisons with established workflows are needed to assess wider scientific use.

The benchmark was small and used two models from one provider within a single local software environment. Each comparison included 24 runs per arm across eight scenario types, yielding wide confidence intervals. The paired difference was 16.7 percentage points for V4-Flash (exact McNemar $p = 0.219$) and 20.8 percentage points for V4-Pro ($p = 0.0625$). All five discordant Pro pairs favored PaleoRigor, but neither test met the 0.05 significance threshold. These results met the engineering criterion without establishing general superiority. Both earlier rounds achieved 75.0% and informed development, so v3 and v4 cannot provide independent validation of v5. Larger preregistered evaluations should include other providers, environments, unfamiliar errors, and scientific claims.

The control arm is an ablation, not an external baseline. Because the PaleoRigor planning contract names the four boundary reason codes explicitly, the 12 of 12 boundary result shows that the encoded control layer is applied consistently; it does not show that the system infers scientific boundaries independently. Separating those two effects would require a third arm that receives the boundary rules but not the workflow-staging rules, which was not run. Comparison with manual command-line practice, fixed standard operating procedures, or established workflow engines such as Snakemake and Nextflow also remains outstanding. Until such comparisons are available, these results support the internal design claim rather than a claim of advantage over current practice.

Conclusions

PaleoRigor helps paleobiologists inspect the path from a research request and its source files to recorded analysis outputs. The frozen release passed 23 of 24 held-out runs, including all 12 boundary decisions, compared with 19 of 24 for the ablated arm. Six public sequencing records matched repository read counts and compressed-file sizes, and the removal of 114 duplicate table rows was documented. These findings support its use for workflow review and evidence preparation within the tested scope. Independent benchmarks, comparisons with established workflow practice, controlled contamination tests, ancient-DNA-specific checks, and community-maintained skills are needed to establish broader applicability. By preserving the context of computational decisions, PaleoRigor provides a basis for specialist review of fragile microbial evidence.

Methods

Agent workflow representation

A user request was represented as an ordered workflow W containing n executable steps:

$$W = \{s_1, s_2, \dots, s_n\}$$

Each step s_i was defined by a skill identifier k_i , input references I_i , output declarations O_i , parameters θ_i , and a natural-language reason r_i :

$$s_i = (k_i, I_i, O_i, \theta_i, r_i)$$

The workflow representation records source files, intended outputs, parameters, and reasons before any analysis runs. Each skill identifier refers to a predefined module that can be inspected independently of the planning model. This extends established reproducible-workflow principles with a natural-language planning and review stage [16–18, 30]. Figure 1 relates the representation to Plan, Review, Execute, and Outputs; Table 1 identifies responsibility at each stage.

Workflow validation

The validator evaluated a proposed workflow W against the available uploaded files F and the available skill registry K :

$$V(W, F, K) \rightarrow \{0, 1\}$$

Acceptance required three conditions: each step used a registered skill, each input referred to an uploaded file or an earlier output, and input formats matched the skill:

$$I_i \subseteq F \cup O_{<i}$$

$$\text{format}(I_i) \in \text{compatible}(k_i)$$

These checks identify missing outputs, unresolved references, and incompatible file types before execution. Supplementary Table S5 links the observed errors to the corresponding checks. Because valid references and formats do not establish that a scientific claim is justified, requests exceeding the permitted scope were evaluated separately.

Execution environment and implemented skills

Validated steps were executed through predefined local skills. Each skill wrapper connected a workflow step to a tool and recorded inputs, parameters, outputs, warnings, and errors. Table skills standardized peptide CSV files, checked sequences, removed duplicates, and exported cleaned tables. Sequencing skills supported file detection, pairing checks, FastQC, MultiQC, SeqKit statistics, and source audits. External tools ran in configured environments; missing software was reported to the user without automatic installation. Figures 3–5 show outputs from these operations [20–22], and Supplementary Table S6 records the software environment.

Human-in-the-loop checkpoints

Experts reviewed the analysis at three stages. At intake, they checked whether the request addressed the scientific question. Before execution, they approved, revised, or rejected the proposed workflow

and its dependencies. Afterwards, they assessed final and intermediate outputs to decide whether further biological analysis was justified. Figure 1 and Supplementary Table S4 show the decisions assigned to each checkpoint.

Scope and non-goals

PaleoRigor was designed to support workflow review, execution, and documentation. Its checks cover repository metadata, file compatibility, workflow references, quality-control outputs, and run records. Metadata agreement alone cannot establish byte-level identity, ancient-DNA authenticity, contamination sources, microbial ecology, or the validity of a retracted claim. Experts remain responsible for archaeological interpretation. Figure 5D illustrates the scope of a permitted conclusion, and Task 6 tests adherence to that scope.

Engineering benchmark protocol

Each benchmark round used a fixed paired design with eight scenarios: four supported workflows and four boundary requests. Three repetitions per scenario in each arm produced 24 runs per arm and 48 API calls per round. The arms used the same model configuration, with calls made in alternating order. Both arms received the same workflow schema, blocked-decision schema, uploaded-file summaries and skill catalogue; this shared block was byte-identical at 35,682 characters. The ablated arm received, in addition, only a four-sentence planner instruction of 247 characters. The PaleoRigor arm received the full planning contract of 40,347 characters, comprising thirty rules governing workflow stages, file compatibility, prerequisites and declared outputs, one worked example workflow, and six control-layer rules that state the four boundary reason codes together with the limits on scientific claims. Supported workflows were executed locally; boundary requests were evaluated as decisions and were never executed.

PaleoRigor passed 18 of 24 runs in each of v3 and v4, which were retained as failed qualification rounds. Their failures informed development and were not reclassified as confirmatory successes. Before v5, we corrected output-name mapping, FASTA/FASTQ suffix handling, peptide-statistics declarations, chart aliases, and metadata-only sample-sheet planning. We froze the v5 manifest, inputs, prompts, scoring rules, threshold, and code before the first API call. V5 used new files, identifiers, values, and wording, with a fixed sample size and no selective reruns or replacement of failures.

For each run r and required criterion d , $q_{r,d}$ was 1 when the run passed the stated rule and 0 when it failed. Supported-run criteria covered parsing, required and forbidden skills, production workflow validation, and local execution. Boundary runs required a valid blocked response with the prespecified reason code. A run passed only when every relevant rule passed:

$$R_r = \prod_{d \in D_r} q_{r,d}$$

For task t with n_t repeated runs, the pass rate was calculated as:

$$\hat{p}_t = (1/n_t) \sum_{r \in t} R_r$$

Strict-success proportions are reported with two-sided Wilson 95% confidence intervals. The prespecified v5 criterion required at least 22 of 24 PaleoRigor runs to pass, exceeding 90%. Paired differences were reported in percentage points and assessed with an exact two-sided McNemar test;

this secondary comparison had limited power with 24 pairs. Each v5 label was independently recomputed from its retained response, workflow or blocked decision, validation result, and execution record. All 48 labels matched. Manifests, scores, summaries, and Figure 2 source data are retained under analysis/benchmark_v3, analysis/benchmark_v4, analysis/benchmark_v5, and Final data/manuscript_figures/benchmark_v5.

For the second-model check, we reused the frozen v5 design and changed the requested model from DeepSeek-V4-Flash to DeepSeek-V4-Pro. The eight scenarios, three repetitions, two arms, prompts, temperature (0), thinking mode, and JSON response mode were unchanged. Validation, scoring, local execution, and the fixed denominator were also retained. The Pro hypothesis and decision rule were committed before the first formal call. The criterion required at least 22 of 24 Pro/PaleoRigor successes and more successes than Pro/raw. Recomputing all 48 Pro labels from retained records confirmed every label; no API call failed. The previously observed Flash results provided descriptive context only.

A third model configuration, deepseek-v4-flash-vision-exp, was also run against the frozen v5 design on 21 August 2026, but the experiment was not completed: only the ablated arm was executed (17 of 24 successes, 70.8%; Wilson 95% confidence interval, 50.8–85.1%). No matched PaleoRigor arm was run, so this configuration yields no paired comparison and contributes no result to the present manuscript. Its manifest, run records and summary are retained in the repository and are reported here so that the retained evaluation materials can be reconciled with the results presented above.

Planning-model configuration

All planning calls used the OpenAI-compatible chat-completions endpoint of the model provider. Each request set the sampling temperature to 0, enabled the provider's extended-reasoning mode, and required a JSON object response, with a 120 s per-request timeout. Transport and HTTP errors were retried up to twice with exponential backoff capped at 4 s. When a returned object failed schema validation, the planner issued exactly one repair call that resupplied the original system prompt together with the validation errors and the invalid object, and instructed the model to correct the JSON without changing the intended task; a second failure aborted the run. In the reported evaluations no request required a retry and no completion required a repair call: all 48 v5 runs and all 48 second-model runs succeeded on the first attempt. The provider exposes no sampling seed, so individual runs are not bit-reproducible.

The frozen v5 evaluation used deepseek-v4-flash and ran on 19 August 2026 between 02:34 and 02:50 UTC at repository commit 4be5f2f. The preregistered second-model check used deepseek-v4-pro and ran on 19 August 2026 between 06:52 and 07:14 UTC at commit 728a226. The only change to the evaluation code between those commits was the addition of per-model configuration loading; the scenarios, prompts, schemas, scoring rules and agent source were unchanged. Each run stores a provenance record containing the arm, start and end times, the model string reported by the interface, token usage, latency, attempt count, repair-call count and the repository commit. The complete system prompts for both arms are reproduced in Supplementary File 4.

Source-level agreement metrics

Read-count and compressed-file-size agreement were calculated against public ENA/SRA metadata:

$$\Delta\text{reads} = N_{\text{agent}} - N_{\text{public}}$$

$$R_{\text{bytes}} = B_{\text{local}} / B_{\text{public}}$$

Agreement at the measured metadata level required $\Delta\text{reads} = 0$ and $R_{\text{bytes}} = 1$. Here, Δreads denotes the difference between local and public read counts; R_{bytes} is the ratio of local compressed FASTQ size to public fastq_bytes. Run records retained accessions, library layout, and mate organization. R1 and R2 denote paired-end mate files, while single denotes a single-end file. Figure 4 applies these measures to the sequencing panel, and Figure 5B applies them to the retraction-associated case. They measure repository agreement rather than biological validity.

ENA's filereport service publishes an MD5 checksum for each submitted FASTQ file. Because the public sequencing files were not retained after analysis, checksums were not recomputed here, and agreement is therefore reported at the metadata level only. Δreads and R_{bytes} cannot exclude different content of identical read count and compressed length. A cryptographic checksum comparison should be the primary source-identity metric in future releases, with Δreads and R_{bytes} retained as secondary descriptors.

FASTQ quality-control summaries

FASTQ summaries included GC percentage, mean read length, and the fraction of bases recorded as N. For a FASTQ file with N reads and read lengths L_j , the measures were:

$$\text{GC}(\%) = (G + C) / (A + T + G + C + N) \times 100$$

$$L = (1/N) \sum_j L_j$$

$$f_N = N_{\text{bases},N} / N_{\text{bases},\text{total}}$$

FastQC and MultiQC generated standard quality-control files [20, 21]. SeqKit produced short file-level summaries used in the figures [22]. GC content and mean read length in Figures 4C and 5C describe the files; they are not evidence of ancient-DNA authenticity.

Dataset-level evaluations and rigor criteria

The evaluation comprised three versioned benchmark rounds and three dataset-level case studies (Table 4). V3 and v4 informed development, whereas v5 evaluated the frozen release on held-out inputs. The peptide table tested whether data changes were recorded, and the six-record FASTQ panel tested agreement with public metadata. The retraction-associated case examined an expert-defined limit on interpretation. Outcomes included strict workflow success, correct blocking decisions, duplicate removal, table structure, read counts, compressed-file sizes, and descriptive quality-control measures.

Table 4. Datasets and measured results used in the evaluation. The table lists each input, its public source, the PaleoRigor task, and the measured output or required boundary. Public accessions identify the source data. The reported values show what was reproduced without making a biological claim.

Evaluation dataset	Input and original source	PaleoRigor task	Quantitative readout or expected boundary
Peptide CSV curation	Headerless local Validation.csv peptide table; 5,810 rows. Project repository [31]	Normalize columns, validate peptide sequences, remove duplicate sequences, compute summaries and export a cleaned table.	5,696 non-redundant records exported; 114 duplicate sequence entries removed; label balance and peptide-length distribution retained as figure checks.
Public FASTQ source-QC panel	Six public sequencing records: ERR15682270, ERR10114877, ERR3250149, ERR10114867, ERR10114861 and ERR15682267. Original ENA accessions listed above [6, 7]	Confirm local files match intended ENA/SRA records before downstream analysis; generate FastQC, MultiQC and seqkit summaries.	PaleoRigor-derived read counts matched public metadata with $\Delta\text{reads} = 0$; local compressed-file ratios were $R\text{bytes} = 1.000$; GC content and mean read length were reported as QC metrics.
Retraction-associated boundary audit	Paired FASTQ record SRR29088443 from SRP508771 / PRJNA1111407; 53,571 reads per mate and 26,517,645 total bases. Original SRA and BioProject accessions listed above [6]	Test whether a problematic literature-linked record can be converted into a source-data audit without validating the retracted biological claim.	R1 reads, R2 reads, total bases, R1 file size and R2 file size matched the public record exactly; biological interpretation was treated as expert-gated.
Engineering benchmark	Versioned v3–v5 benchmark; each round used eight scenarios, three repeats, and matched PaleoRigor/raw-model arms (24 runs per arm).	Test strict workflow success, local execution, input/scientific boundaries, and the effect of the PaleoRigor control layer under fixed model settings.	V3: 18/24; v4: 18/24; frozen v5: PaleoRigor 23/24 (95.8%) and the ablated arm 19/24 (79.2%). V5 boundary decisions: 12/12.

Output organization, public website and packaged desktop applications

Each local run created a dated result folder. `final_outputs` contained the files requested by the user. `step_outputs` contained intermediate files. ResearchAgent Records contained workflows, manifests, checksums, times, and reports. This structure keeps both the requested result and the evidence used to produce it. The public website [29] explains the method and installation process; it does not accept research data or run workflows. Real analyses run locally in the browser interface, where users review the workflow, approve execution, monitor progress, and inspect the report. For the macOS research prototype, this interface is launched by a self-contained Apple Silicon application targeting macOS 13 or later. The application bundles the PaleoRigor backend together with FastQC 0.12.1, MultiQC 1.35, SeqKit 2.13.0, SeqTk 1.5-r133, Samtools 1.23.1, BWA 0.7.19-r1273, Bowtie2 2.5.5, and their required runtimes. Users therefore do not need to install VS Code, Python, Conda, Homebrew, Java, or these seven tools separately. The backend binds only to the local loopback interface, uses a temporary per-launch authentication token, and stores model API credentials in macOS Keychain rather than in the application bundle. The Windows installer exposes the same local browser interface from the same backend source,

bundles the same seven tools with an Eclipse Temurin 21.0.8+9 runtime, and stores model credentials in Windows Credential Manager instead.

Reproducibility bundle and supplementary materials

Each case retained final outputs, intermediate files, and run records. Public sequencing files were not duplicated in the repository; their accessions and metadata identify the ENA/SRA records for retrieval. The repository contains processed outputs, workflows, quality-control summaries, manifests, checksums, and plotting scripts. It also includes the v3–v5 benchmark manifests, the preregistered V4-Pro check, run-level scores, summaries, verification reports, and Figure 2 source data. Supplementary Tables S1–S3 link the reported results to these records.

The supplementary package organizes six evidence tables alongside workflows, run reports, source manifests, and figure data with plotting scripts. It distinguishes public records from local inputs, intermediate outputs, final results, and manuscript figures, allowing readers to trace reported values to their supporting files.

Supplementary Information

The Supplementary Information contains six evidence tables and four file groups. Supplementary Table S1 links each case to final outputs, intermediate files, and run records. Supplementary Tables S2–S6 cover case evidence, claim-to-evidence links, expert checkpoints, error controls, and software settings. Supplementary File 1 contains workflows, reports, manifests, and checksums. Supplementary File 2 contains figure data and scripts. Supplementary File 3 contains example final_outputs and step_outputs. Supplementary File 4 contains the verbatim system prompts sent to both benchmark arms, together with the byte-identical shared contract and the content unique to each arm. Together, these materials support Figures 1–5 and Tables 1–4 without making the main text too long.

Supplementary Table S1. Links to final outputs, intermediate files, and run records. final_outputs contains requested result files. step_outputs contains files produced by each workflow step. ResearchAgent Records contains workflow JSON, manifests, timestamps, checksums, and reports. Each row links a case or sequencing record to the repository folder used to review its evidence trail.

Validation item	Result folder	Repository location	Evidence role
Case 1: peptide CSV curation	Final data/1	PaleoRigor repository path: Final data/1 [31]	Cleaned CSV, peptide statistics, charts, normalization/validation/deduplication step outputs, workflow manifest and report. Subfolders at this repository location contain final_outputs, step_outputs and ResearchAgent Records.
Case 2a: ERR15682270 R1	Final data/2	PaleoRigor repository path: Final data/2 [31]	FASTQ file typing, FastQC/MultiQC outputs, quality-gate report and run records for mate R1. Subfolders at this repository location contain final_outputs, step_outputs and ResearchAgent Records.
Case 2b: ERR15682270 R2	Final data/3	PaleoRigor repository path: Final data/3 [31]	FASTQ file typing, FastQC/MultiQC outputs, quality-gate report and run records for mate R2. Subfolders at this repository location contain final_outputs, step_outputs and ResearchAgent Records.
Case 2c: ERR10114877 paired FASTQ	Final data/group 4-5 new	PaleoRigor repository path: Final data/group 4-5 new [31]	Paired-end FASTQ QC outputs and source-record audit records after rerun with corrected inputs. Subfolders at this repository location contain final_outputs, step_outputs and ResearchAgent Records.
Case 2d: ERR3250149 single FASTQ	Final data/6	PaleoRigor repository path: Final data/6 [31]	Single-end FASTQ QC outputs, source-record checks and local run records. Subfolders at this repository location contain final_outputs, step_outputs and ResearchAgent Records.
Case 2e: ERR10114867 paired FASTQ	Final data/group 7 new	PaleoRigor repository path: Final data/group 7 new [31]	Paired-end FASTQ QC outputs and source-record audit records after rerun with corrected inputs. Subfolders at this repository location contain final_outputs, step_outputs and ResearchAgent Records.

Validation item	Result folder	Repository location	Evidence role
Case 2f: ERR10114861 paired FASTQ	Final data/8	PaleoRigor repository path: Final data/8 [31]	Paired-end FASTQ QC outputs, metadata checks and run records. Subfolders at this repository location contain final_outputs, step_outputs and ResearchAgent Records.
Case 2g: ERR15682267 paired FASTQ	Final data/9	PaleoRigor repository path: Final data/9 [31]	Paired-end FASTQ QC outputs, metadata checks and run records. Subfolders at this repository location contain final_outputs, step_outputs and ResearchAgent Records.
Case 3: retraction-associated audit	Final data/retracted_result	PaleoRigor repository path: Final data/retracted_result [31]	Boundary-aware source-data audit, seqkit/FastQC/MultiQC summaries, intermediate step outputs and run records. Subfolders at this repository location contain final_outputs, step_outputs and ResearchAgent Records.
Engineering benchmark	analysis/ benchmark_v3; analysis/benchmark_v4 ; analysis/benchmark_v5	PaleoRigor repository path: analysis/benchmark_v5 [31]	Versioned manifests, preregistrations, run-level scores, summary JSON, verification report, and Figure 2 source data; complete raw run bundles are retained in the project archive.

Supplementary Table S2. Evidence files for each validation workflow. The table links each case to its natural-language request, workflow records, final outputs, and intermediate files.

Case	Task and workflow evidence	Final-output evidence	Intermediate evidence
Case 1 peptide CSV curation	Natural-language peptide-table curation request; workflow JSON and report in ResearchAgent Records.	cleaned CSV, statistics and charts; GitHub result-folder URL listed in Supplementary Table S1	normalization, validation, statistics, deduplication and export step outputs; GitHub result-folder URL listed in Supplementary Table S1
Case 2 public FASTQ source-QC panel	FASTQ source-check and QC requests for ERR15682270, ERR10114877, ERR3250149, ERR10114867, ERR10114861 and ERR15682267.	FastQC/MultiQC summaries and metadata JSON files; GitHub result-folder URLs listed in Supplementary Table S1	per-record step_outputs and ResearchAgent Records; GitHub result-folder URLs listed in Supplementary Table S1
Case 3 retraction-associated audit	Boundary-aware audit request for SRR29088443; workflow report documents that source-data reproducibility rather than biological validity was tested.	audit summaries and QC outputs; GitHub result-folder URL listed in Supplementary Table S1	file detection, FastQC, MultiQC, seqkit and run records; GitHub result-folder URL listed in Supplementary Table S1
Engineering benchmark	Versioned v3–v5 manifests, preregistrations, raw completions, workflows or blocked decisions, validation/execution records, and independent scoring outputs.	v5 run_level.csv, summary.json, verification_report.md, Figure 2 source data, and SVG/PDF/PNG/TIFF exports.	V3–v5 records are under analysis/benchmark_v3 , analysis/benchmark_v4 , analysis/benchmark_v5 , and Final data/manuscript_figures/benchmark_v5.

Supplementary Table S3. Links between manuscript claims and evidence. Each claim is paired with the file or record that supports it.

Manuscript claim	Primary evidence file or record	Figure/Table location	Repository route
Peptide table curation removed 114 duplicate sequence entries and exported 5,696 non-redundant records.	cleaned_Validation.csv, peptide_statistics.json and deduplication step outputs.	Figure 3; Table 4	Case 1 result-folder URL in Supplementary Table S1
Public FASTQ source records matched local inputs at the read-count and compressed-file-size level.	fastqc_run_metadata.json, file_type_report.json, MultiQC output and run records for Cases 2a-2g.	Figure 4; Table 4 and Supplementary Table S1	Case 2a-2g result-folder URLs in Supplementary Table S1
The retraction-associated case reproduced source-level metadata without validating the retracted biological conclusion.	seqkit_stats.tsv, fastqc_run_metadata.json, workflow report and boundary-aware audit figure source data.	Figure 5; Table 4 and Supplementary Tables S1–S2	Case 3 result-folder URL in Supplementary Table S1
The system separates user-facing final outputs from intermediate step outputs and run records.	Timestamped result folders containing final_outputs, step_outputs and ResearchAgent Records.	Supplementary Table S1	Repository result folders listed in Supplementary Table S1
Workflow validation prevents unsupported execution such as missing step outputs or incompatible file-type operations.	Workflow validation errors and development failure cases documented in run records and failure-mode audit.	Table 2 and Supplementary Table S5	ResearchAgent Records listed through Supplementary Table S1 result-folder URLs
The frozen final release exceeded the prespecified strict-success threshold, while earlier qualification failures remained visible.	v3–v5 manifests and summaries; v5 run_level.csv, summary.json, verification_report.md, and retained execution records.	Figure 2; Table 3	analysis/ benchmark_v3, analysis/benchmark_v4 , analysis/benchmark_v5 , and Figure 2 source-data folders

Supplementary Table S4. Expert checkpoints. The table shows where expert review occurs and which error each checkpoint is intended to prevent.

Workflow stage	PaleoRigor action	Human decision	Failure prevented
Task intake	Converts natural-language request into structured workflow steps with declared inputs, outputs and parameters.	Check whether the planned task matches the scientific question before execution.	Wrong task framing or use of an irrelevant skill.
Pre-execution validation	Checks file references, step-output dependencies and input-format compatibility.	Approve, revise or reject the proposed workflow.	Hallucinated step references, CSV/FASTQ mismatch or unsupported operation.
Local execution	Runs only predefined skills and reports progress, warnings and missing environment dependencies.	Decide whether to install missing tools or stop the run.	Silent auto-installation, uncontrolled code execution or hidden dependency failure.
Post-run review	Exports final_outputs, step_outputs and ResearchAgent Records.	Interpret outputs and decide whether downstream biological analysis is justified.	Treating QC/source checks as biological validation.
Boundary cases	Flags source-level reproducibility separately from claim-level interpretation.	Gate contested or contamination-sensitive conclusions for domain review.	Overclaiming from low-biomass or retraction-associated records.

Supplementary Table S5. Errors and implemented controls. Development failures were kept because they show why each safety check was needed.

Observed failure mode	How it appeared	Implemented control	Evidence route
Missing step-output reference	Generated workflow referred to outputs such as step_02:qc_zip or step_01/normalized that the validator could not resolve.	Strict step-output reference validation before execution.	workflow validation run records
Wrong input type for requested operation	FASTQ QC was requested for a CSV upload, or gzip decompression was proposed for an already FASTQ-classified file.	File-type detection and skill compatibility checks block incompatible workflows.	workflow records and file-type compatibility checks
Quality gate stopping on expected duplicates	The peptide validation file contained duplicate sequences, causing a strict gate to fail before deduplication.	Deduplication and quality-gate order clarified; warnings can be audited before downstream use.	Case 1 step_outputs and deduplication records
Tool-output directory assumption	FastQC initially failed when its output directory was not created before execution.	Skill wrapper creates required output directories and records tool metadata.	FASTQ final_outputs and tool metadata
User prompt/file-name mismatch	A rerun failed when the natural-language prompt still named files from a previous group.	Workflow review exposes uploaded-file references before local execution.	corrected rerun records

Supplementary Table S6. Software and environment details. The table lists the local software used to generate the evaluation outputs and figures. The final release should fix these versions in a tagged manifest.

Component	Version, environment and original URL	Role in study	Release note
PaleoRigor project	Repository: PaleoRigor GitHub repository [31]; versioned release tag and commit to be assigned before submission.	Workflow planning, validation, local skill execution and output organization.	To be archived as a tagged GitHub release and DOI-linked snapshot before submission.
Python runtime for manuscript scripts	Python 3.13.9 in project virtual environment; Python: Python website [32]; pandas: pandas website [33]; matplotlib: Matplotlib website [34]	Figure source-data processing and manuscript figure generation.	Used with pandas 2.3.3 and matplotlib 3.11.0.
FastQC	FastQC v0.12.1; software reference [20]	FASTQ quality-control reports for source-QC and boundary-audit cases.	Called through predefined fastq_qc skill; reports preserved in final_outputs.
MultiQC	MultiQC v1.35 in multiqc_env; software reference [21]	Aggregation of FastQC reports into summary HTML/metadata outputs.	Missing environment is reported rather than silently installed.
SeqKit	SeqKit v2.13.0 in base environment; software reference [22]	Compact FASTQ statistics and source-level summaries.	Used for read/base/length checks where available.
Repository and archive plan	GitHub repository: PaleoRigor GitHub repository [31]; planned archive: [Zenodo DOI pending]	Stable access to code, outputs, step files, run records and figure source data.	Replace mutable main-branch links with release-specific links before final submission.

Supplementary note: Figure generation and image provenance

All figures were produced from author-created vector schematics or Python-based plots of analytical outputs. The graphical abstract incorporates an openly licensed archaeological photograph with attribution. No figure was generated with a text-to-image model. Figure 2 was generated in Python from the retained v3–v5 benchmark summaries and run-level records. Its source-data CSV, plotting script, and editable SVG, PDF, PNG, and TIFF exports are preserved with the supplementary analysis materials.

Declarations

List of abbreviations

aDNA, ancient DNA; API, application programming interface; bp, base pairs; CI, confidence interval; ENA, European Nucleotide Archive; GC, guanine–cytosine; JSON, JavaScript Object Notation; LLM, large language model; QC, quality control; SOP, standard operating procedure; SRA, Sequence Read Archive.

Ethics approval and consent to participate

Not applicable. This study performed secondary computational analyses of public repository records and small example files and did not recruit participants, collect new human or animal material, or generate new human-subject data.

Consent for publication

Not applicable.

Availability of data and materials

The datasets analysed in this study are public repository records or small example files. Public sequence inputs are identified by accession in Table 4. Evaluation manifests, run-level scores, workflows, blocked decisions, execution records, processed outputs, figure source data, and plotting scripts are available in the PaleoRigor repository [31]. Large public FASTQ files were not duplicated in the repository; their accession identifiers enable retrieval from ENA or SRA [6, 7].

Project name: PaleoRigor. Project home page: PaleoRigor project website [29]. Source code: GitHub repository [31]. Desktop research applications, installation instructions, version information, and SHA-256 checksums: <https://github.com/Verture-Liu/2026summerproject/tree/main/paleorigor>. Archived version: [Zenodo DOI and release tag to be inserted before submission]. Supported packaged platforms: Apple Silicon macOS 13 or later, and Windows 10/11 x64. The reported evaluation used the macOS application; the Windows build was verified by its automated build and tool-identity checks only and produced no reported result. Programming language: Python 3.13.9. License: MIT. Redistributed third-party tools retain their own licenses, several of which are GPL-3.0; these are listed with their source obligations in THIRD_PARTY_NOTICES.md in the repository. Restrictions on non-academic use: none.

Competing interests

The authors declare that they have no competing interests.

Funding

[All funding sources and the role of each funding body in study design, data analysis, interpretation, and manuscript preparation must be confirmed by the authors before submission.]

Authors' contributions

[Author initials and individual contributions must be completed after the final author list is confirmed. All authors must read and approve the final manuscript.]

Acknowledgements

[Supervisors, collaborators, and contributors who do not meet authorship criteria must be listed here after permission is obtained. If none, replace this text with 'Not applicable'.]

References

1. Adler CJ, Dobney K, Weyrich LS, Kaidonis J, Walker AW, Haak W, et al. Sequencing ancient calcified dental plaque shows changes in oral microbiota with dietary shifts of the Neolithic and Industrial revolutions. *Nat Genet.* 2013;45:450–455. doi:10.1038/ng.2536.
2. Warinner C, Rodrigues JFM, Vyas R, Trachsel C, Shved N, Grossmann J, et al. Pathogens and host immunity in the ancient human oral cavity. *Nat Genet.* 2014;46:336–344. doi:10.1038/ng.2906.
3. Warinner C, Herbig A, Mann A, Fellows Yates JA, Weiß CL, Burbano HA, et al. A robust framework for microbial archaeology. *Annu Rev Genomics Hum Genet.* 2017;18:321–356. doi:10.1146/annurev-genom-091416-035526.
4. Weyrich LS, Dobney K, Cooper A. Ancient DNA analysis of dental calculus. *J Hum Evol.* 2015;79:119–124. doi:10.1016/j.jhevol.2014.06.018.
5. Weyrich LS, Duchene S, Soubrier J, Arriola L, Llamas B, Breen J, et al. Neanderthal behaviour, diet, and disease inferred from ancient DNA in dental calculus. *Nature.* 2017;544:357–361. doi:10.1038/nature21674.
6. Leinonen R, Sugawara H, Shumway M. The Sequence Read Archive. *Nucleic Acids Res.* 2011;39:D19–D21. doi:10.1093/nar/gkq1019.
7. O'Cathail C, Ahamed A, Burgin J, Cummins C, Devaraj R, Gueye K, et al. The European Nucleotide Archive in 2024. *Nucleic Acids Res.* 2025;53:D49–D55. doi:10.1093/nar/gkae975.
8. Briggs AW, Stenzel U, Johnson PLF, Green RE, Kelso J, Prüfer K, et al. Patterns of damage in genomic DNA sequences from a Neandertal. *Proc Natl Acad Sci USA.* 2007;104:14616–14621. doi:10.1073/pnas.0704665104.
9. Cooper A, Poinar HN. Ancient DNA: do it right or not at all. *Science.* 2000;289:1139. doi:10.1126/science.289.5482.1139b.
10. Dabney J, Meyer M, Pääbo S. Ancient DNA damage. *Cold Spring Harb Perspect Biol.* 2013;5:a012567. doi:10.1101/cshperspect.a012567.
11. Gilbert MTP, Bandelt H-J, Hofreiter M, Barnes I. Assessing ancient DNA studies. *Trends Ecol Evol.* 2005;20:541–544. doi:10.1016/j.tree.2005.07.005.

12. Skoglund P, Northoff BH, Shunkov MV, Derevianko AP, Pääbo S, Krause J, et al. Separating endogenous ancient DNA from modern day contamination in a Siberian Neandertal. *Proc Natl Acad Sci USA*. 2014;111:2229–2234. doi:10.1073/pnas.1318934111.
13. Jónsson H, Ginolhac A, Schubert M, Johnson PLF, Orlando L. mapDamage2.0: fast approximate Bayesian estimates of ancient DNA damage parameters. *Bioinformatics*. 2013;29:1682–1684. doi:10.1093/bioinformatics/btt193.
14. Schubert M, Ginolhac A, Lindgreen S, Thompson JF, Al-Rasheid KAS, Willerslev E, et al. Improving ancient DNA read mapping against modern reference genomes. *BMC Genomics*. 2012;13:178. doi:10.1186/1471-2164-13-178.
15. Schubert M, Lindgreen S, Orlando L. AdapterRemoval v2: rapid adapter trimming, identification, and read merging. *BMC Res Notes*. 2016;9:88. doi:10.1186/s13104-016-1900-2.
16. Di Tommaso P, Chatzou M, Floden EW, Barja PP, Palumbo E, Notredame C. Nextflow enables reproducible computational workflows. *Nat Biotechnol*. 2017;35:316–319. doi:10.1038/nbt.3820.
17. Köster J, Rahmann S. Snakemake—a scalable bioinformatics workflow engine. *Bioinformatics*. 2012;28:2520–2522. doi:10.1093/bioinformatics/bts480.
18. Sandve GK, Nekrutenko A, Taylor J, Hovig E. Ten simple rules for reproducible computational research. *PLoS Comput Biol*. 2013;9:e1003285. doi:10.1371/journal.pcbi.1003285.
19. Yao S, Zhao J, Yu D, Du N, Shafran I, Narasimhan K, et al. ReAct: synergizing reasoning and acting in language models. *arXiv*. 2022. <https://arxiv.org/abs/2210.03629>.
20. Andrews S. FastQC: a quality control tool for high throughput sequence data. Babraham Bioinformatics. <https://www.bioinformatics.babraham.ac.uk/projects/fastqc/>. Accessed 19 Aug 2026.
21. Ewels P, Magnusson M, Lundin S, Käller M. MultiQC: summarize analysis results for multiple tools and samples in a single report. *Bioinformatics*. 2016;32:3047–3048. doi:10.1093/bioinformatics/btw354.
22. Shen W, Le S, Li Y, Hu F. SeqKit: a cross-platform and ultrafast toolkit for FASTA/Q file manipulation. *PLoS One*. 2016;11:e0163962. doi:10.1371/journal.pone.0163962.
23. Gao W, Wang X, Shi Y, Wu G, Zhou M, Lin X. Retracted article: predictable regulation of gut microbiome in immunotherapeutic efficacy of gastric cancer. *Genes Immun*. 2025;26:1–8. doi:10.1038/s41435-024-00306-2.
24. Gao W, Wang X, Shi Y, Wu G, Zhou M, Lin X. Retraction note: predictable regulation of gut microbiome in immunotherapeutic efficacy of gastric cancer. *Genes Immun*. 2026;27:384. doi:10.1038/s41435-026-00397-z.
25. Davis NM, Proctor DM, Holmes SP, Relman DA, Callahan BJ. Simple statistical identification and removal of contaminant sequences in marker-gene and metagenomics data. *Microbiome*. 2018;6:226. doi:10.1186/s40168-018-0605-2.

26. Eisenhofer R, Minich JJ, Marotz C, Cooper A, Knight R, Weyrich LS. Contamination in low microbial biomass microbiome studies: issues and recommendations. *Trends Microbiol.* 2019;27:105–117. doi:10.1016/j.tim.2018.11.003.
27. Karstens L, Asquith M, Davin S, Fair D, Gregory WT, Wolfe AJ, et al. Controlling for contaminants in low-biomass 16S rRNA gene sequencing experiments. *mSystems.* 2019;4:e00290-19. doi:10.1128/mSystems.00290-19.
28. Salter SJ, Cox MJ, Turek EM, Calus ST, Cookson WO, Moffatt MF, et al. Reagent and laboratory contamination can critically impact sequence-based microbiome analyses. *BMC Biol.* 2014;12:87. doi:10.1186/s12915-014-0087-z.
29. PaleoRigor project website. <https://verture-liu.github.io/2026summerproject/>. Accessed 19 Aug 2026.
30. Wilkinson MD, Dumontier M, Aalbersberg IJ, Appleton G, Axton M, Baak A, et al. The FAIR Guiding Principles for scientific data management and stewardship. *Sci Data.* 2016;3:160018. doi:10.1038/sdata.2016.18.
31. PaleoRigor source code and reproducibility materials. GitHub. <https://github.com/Verture-Liu/2026summerproject>. Accessed 19 Aug 2026.
32. Python Software Foundation. Python. <https://www.python.org/>. Accessed 19 Aug 2026.
33. The pandas development team. pandas. <https://pandas.pydata.org/>. Accessed 19 Aug 2026.
34. The Matplotlib development team. Matplotlib. <https://matplotlib.org/>. Accessed 19 Aug 2026.